Pipeline de Observabilidad End-to-End con OpenTelemetry

Instrumentacion de microservicios, correlacion cross-signal y analisis de overhead

Laboratorio: Pipeline OpenTelemetry con Jaeger y Prometheus

Programa: Maestria en Arquitectura de Software — Universidad de La Sabana

Asignatura: Observabilidad en Ambientes Productivos

Autor: Julian Miranda

Fecha: 14/10/2026

---

1. Introduccion y objetivo

Este trabajo implementa un pipeline de observabilidad completo sobre una aplicacion de microservicios, capturando las tres senales de telemetria —metricas, logs estructurados y trazas distribuidas— mediante el SDK de OpenTelemetry, y habilitando la navegacion entre las tres a traves de un identificador comun: el `trace_id`.

El objetivo no es unicamente *emitir* telemetria, sino conseguir que las tres senales sean **mutuamente navegables**. Una metrica que muestra un pico de latencia responde *que* ocurrio; la traza asociada explica *donde*; y los logs de esa misma traza revelan *por que*. Sin correlacion, cada senal obliga a reconstruir el contexto a mano, que es precisamente el trabajo que consume el tiempo durante un incidente.

> **Resultado principal:** el pipeline queda verificado de forma automatizada mediante `scripts/verify-pipeline.sh`, que dispara una peticion real y comprueba, consultando las APIs de Jaeger, Loki y Prometheus, que ese mismo `trace_id` aparece en las tres senales y que la traza contiene spans de *ambos* microservicios.

2. Arquitectura implementada

2.1 Flujo de la aplicacion

La aplicacion modela un caso de negocio deliberadamente sencillo pero suficiente para ejercitar todos los mecanismos de observabilidad relevantes: una dependencia HTTP entre servicios y acceso a base de datos en ambos extremos.

```
cliente
  |
  v  GET /order/{id}
service-a (FastAPI :8000)  ---- SQL ---->  PostgreSQL
  |                                           ^
  |  HTTP + header traceparent (W3C)          |
  v                                           |
service-b (FastAPI :8001)  ---- SQL ----------+
```

El salto HTTP de `service-a` a `service-b` es el punto critico: la instrumentacion de HTTPX inyecta la cabecera `traceparent` en la peticion saliente y la de FastAPI la extrae en el servicio receptor. El resultado es que ambos servicios producen spans bajo un **unico** `trace_id`, en lugar de dos trazas inconexas.

2.2 Pipeline de telemetria

```
service-a / service-b
   |
   |  OTLP/gRPC :4317   (trazas + metricas + logs)
   v
OTel Collector
   |  memory_limiter -> resourcedetection -> resource
   |  -> filter/health -> attributes -> batch
   |
   +--> Jaeger      :16686   trazas (UI de referencia)
   +--> Tempo       :3200    trazas (nativo de Grafana)
   +--> Prometheus  :9090    metricas
   +--> Loki        :3100    logs
              |
              v
         Grafana :3000  (correlacion por trace_id)
```

2.3 Las tres senales emitidas

Senal	Mecanismo del SDK	Transporte	Backend
Trazas	TracerProvider + BatchSpanProcessor	OTLP/gRPC	Jaeger, Tempo
Metricas	MeterProvider con dos readers: PeriodicExporting (push) + Prometheus (scrape)	OTLP/gRPC y HTTP :9464	Prometheus
Logs	LoggerProvider + LoggingHandler, y JSON a stdout con trace_id inyectado	OTLP/gRPC	Loki

2.4 Instrumentacion aplicada

Automatica. Tres librerias instrumentadas sin modificar el codigo de negocio: `FastAPI` genera un span de servidor por peticion y extrae el contexto entrante; `HTTPX` genera spans de cliente e inyecta el `traceparent`; `psycopg2` genera un span por sentencia SQL con la consulta como atributo.

Personalizada. Spans que describen la logica de negocio, no solo la mecanica tecnica:

Span	Servicio	Que representa
`order.process`	service-a	Span raiz de la operacion de negocio completa
`fetch.order.db`	service-a	Recuperacion del pedido
`call.service-b.inventory`	service-a	Llamada downstream con propagacion de contexto
`order.compute_total`	service-a	Calculo del importe y decision de surtido
`inventory.lookup.db`	service-b	Consulta de stock
`inventory.validate.stock`	service-b	Regla de negocio sobre el nivel de stock

3. Configuracion del OTel Collector

El Collector actua como punto unico de entrada de las tres senales. Los servicios no conocen ningun backend: solo exportan por OTLP. Ese desacoplamiento permite cambiar de Jaeger a Cloud Trace sin tocar una linea de la aplicacion — exactamente lo que ocurre entre los tres entornos del repositorio.

3.1 Orden de los procesadores

El orden no es arbitrario y condiciona el comportamiento bajo carga:

#	Procesador	Razon de su posicion
1	`memory_limiter`	Debe ir **primero**. Si se colocara despues de `batch`, el lote ya habria reservado la memoria que se intenta limitar.
2	`resourcedetection`	Anade metadatos del entorno antes de que otros procesadores los usen.
3	`resource`	Atributos comunes a las tres senales: son los que permiten filtrarlas por el mismo criterio en Grafana.
4	`filter/health`	Descarta pronto el ruido para no procesarlo ni agruparlo.
5	`attributes`	Control de cardinalidad: cada combinacion de labels es una serie nueva en Prometheus.
6	`batch`	**Ultimo**, justo antes de los exporters, que es donde la agrupacion reduce llamadas de red.

3.2 Una configuracion por entorno

Archivo	Entorno	Exporters
`collector-local.yaml`	Docker Compose	Jaeger, Tempo, Prometheus, Loki
`collector-gcp.yaml`	Cloud Run (sidecar)	Cloud Trace, Cloud Logging, Managed Prometheus
`collector-aws.yaml`	ECS Fargate (sidecar)	X-Ray, CloudWatch Logs, CloudWatch EMF

La estructura de receivers y procesadores es **identica** en los tres archivos; solo cambian los exporters. Esa simetria es intencional: permite validar el comportamiento del pipeline en local y confiar en que la nube se comporta igual, porque la unica variable que cambia es el destino.

> **Alcance de la ejecucion.** El pipeline se ejecuto y verifico de forma completa en el entorno local con Docker Compose. La infraestructura para ambas nubes esta descrita como codigo en `terraform/`, parametrizada y versionada. El despliegue en AWS no se materializo por no disponer de una cuenta; su Terraform se entrega completo pero sin aplicar.

4. Correlacion cross-signal

La correlacion se configura de forma declarativa en el aprovisionamiento de datasources de Grafana, y esta habilitada en **ambos sentidos**:

Salto	Mecanismo	Datasource
Log → Traza	`derivedFields` sobre el trace_id	Loki
Traza → Log	`tracesToLogsV2` con consulta LogQL	Jaeger, Tempo
Traza → Metrica	`tracesToMetrics`	Jaeger, Tempo
Metrica → Traza	`exemplarTraceIdDestinations`	Prometheus

4.1 El trace_id como pivote

Para que el salto funcione, el mismo identificador debe estar presente en las tres senales con el mismo formato. En las **trazas** es intrinseco. En los **logs** se inyecta mediante un formateador JSON que lee el span activo en cada registro. En las **metricas** la union no es por `trace_id` —seria una explosion de cardinalidad inaceptable— sino por los atributos de Resource compartidos, mas los *exemplars*, que enlazan muestras concretas del histograma con la traza que las produjo.

```
{
  "message": "Pedido procesado",
  "order_id": "ORD-0001",
  "trace_id": "d236b3bd43faec1e0e8698df474610c6",
  "span_id":  "06ef8d52c9e6daba",
  "service":  "service-a",
  "level":    "info"
}
```

4.2 Verificacion de la propagacion W3C

La comprobacion decisiva no es que exista un `trace_id`, sino que **sea el mismo a ambos lados del salto HTTP**. El script de verificacion consulta la API de Jaeger y confirma que la traza contiene procesos de `service-a` y de `service-b`: si la propagacion fallara, `service-b` iniciaria una traza independiente y solo apareceran spans de un servicio.

Durante el escenario instrumentado, **10,971** respuestas incluyeron un `trace_id` con formato W3C valido (32 digitos hexadecimales), sobre un total de 12,874 peticiones (85.2 %). El resto corresponde a los endpoints `/orders` y a las consultas de pedidos inexistentes, que por diseno no devuelven `trace_id` en el cuerpo de la respuesta.

5. Analisis de overhead

5.1 Metodologia

La comparacion se realiza entre dos despliegues que ejecutan **la misma imagen y el mismo codigo**. La unica diferencia es la variable de entorno `OTEL_ENABLED`: cuando vale `false`, el modulo de telemetria devuelve implementaciones vacias y los imports del SDK —diferidos a proposito— nunca llegan a ejecutarse.

Un benchmark con dos bases de codigo separadas no mediria solo el costo de OpenTelemetry, sino tambien las diferencias accidentales entre ambas implementaciones. Ambos despliegues reciben ademas limites de recursos identicos (1.0 CPU / 512 MiB); sin esa igualdad, una diferencia de CPU disponible se confundiria con el costo de la instrumentacion.

Las metricas provienen de **dos fuentes independientes**, para no pedirle al proceso instrumentado que se mida a si mismo:

- **Latencia y throughput** — medidos por k6 desde el cliente, fuera del proceso observado.
- **CPU y memoria** — medidos por el receiver `docker_stats` del Collector, que lee las estadisticas del demonio de Docker desde el host.

5.2 Tres regimenes de carga

La medicion se repitio bajo tres niveles de concurrencia. Medir uno solo habria dado una imagen enganosa, porque **el overhead no desaparece ni crece: cambia de forma** segun quede o no CPU libre.

Regimen	CPU en linea base	Overhead en CPU	Perdida de throughput	Memoria
10 VUs — holgado	63.6 %	**+29.9 %**	−5.7 %	+90.9 %
25 VUs — apretado	112.1 %	+5.2 %	−19.4 %	+89.4 %
75 VUs — saturado	119.9 %	+2.4 %	**−24.7 %**	+85.1 %

La relacion entre las dos columnas centrales es **inversa**: conforme la CPU se agota, el overhead medido en consumo se desvanece y reaparece como perdida de capacidad. Es el mismo trabajo adicional manifestandose alli donde queda margen para que se note.

> **Por que importa esta distincion.** Reportar unicamente el escenario saturado habria llevado a la conclusion de que instrumentar cuesta apenas un 2 % de CPU. Eso es falso: la CPU aparece casi igual con y sin OTel no porque la telemetria sea gratuita, sino porque **no hay mas CPU que consumir**. El limite del contenedor actua como techo y enmascara el costo, que se cobra en el throughput.

La memoria, en cambio, sube entre un 85 % y un 91 % en los tres regimenes, con independencia de la carga. Esa estabilidad confirma que se trata de un **costo fijo** del SDK —providers, exporters y buffers de los procesadores por lotes— y no de un consumo proporcional al trafico.

5.3 Resultados

5.3.1 Carga ligera (10 usuarios concurrentes)

La CPU conserva margen libre (64 % de uso en la linea base), por lo que el consumo adicional del SDK es directamente observable. Es la medicion del overhead en si mismo.

- **Usuarios concurrentes:** 10 VUs (rampa de 30 s + meseta de 3m)
- **Peticiones linea base:** 5,516
- **Peticiones instrumentado:** 5,190
- **Limites por contenedor:** 1.0 CPU / 512 MiB (identicos en ambos escenarios)
- **Mezcla de trafico:** 85 % consulta de pedido (A → DB → B → DB), 10 % listado, 5 % pedido inexistente

Metrica	Sin OTel	Con OTel	Diferencia	Overhead
Latencia media	59.49 ms	80.42 ms	20.94 ms	+35.20 %
Latencia p95	122.17 ms	179.16 ms	56.98 ms	+46.64 %
Latencia p99	175.33 ms	280.18 ms	104.85 ms	+59.80 %
Throughput	24.78 req/s	23.37 req/s	-1.41 req/s	-5.71 %
CPU media de los servicios	63.63 %	82.67 %	19.03 pp	+29.91 %
CPU pico de los servicios	77.48 %	103.19 %	25.71 pp	+33.19 %
CPU del OTel Collector	0.95 %	6.37 %	5.42 pp	+568.51 %
Memoria pico de los servicios	102.34 MiB	195.32 MiB	92.99 MiB	+90.87 %

5.3.5 Carga moderada (25 usuarios concurrentes)

El sistema entra en la zona de agotamiento de CPU. El overhead medido en consumo empieza a desaparecer y comienza a reflejarse como perdida de capacidad.

- **Usuarios concurrentes:** 25 VUs (rampa de 30 s + meseta de 4m)
- **Peticiones linea base:** 12,320
- **Peticiones instrumentado:** 9,948
- **Limites por contenedor:** 1.0 CPU / 512 MiB (identicos en ambos escenarios)
- **Mezcla de trafico:** 85 % consulta de pedido (A → DB → B → DB), 10 % listado, 5 % pedido inexistente

Metrica	Sin OTel	Con OTel	Diferencia	Overhead
Latencia media	236.91 ms	371.73 ms	134.81 ms	+56.90 %
Latencia p95	491.39 ms	704.86 ms	213.47 ms	+43.44 %
Latencia p99	647.55 ms	1093.75 ms	446.19 ms	+68.90 %
Throughput	41.72 req/s	33.64 req/s	-8.08 req/s	-19.36 %
CPU media de los servicios	112.12 %	117.90 %	5.78 pp	+5.15 %
CPU pico de los servicios	124.88 %	134.59 %	9.72 pp	+7.78 %
CPU del OTel Collector	3.15 %	3.16 %	0.01 pp	+0.38 %
Memoria pico de los servicios	119.79 MiB	226.88 MiB	107.09 MiB	+89.40 %

5.3.9 Saturacion (75 usuarios concurrentes)

Ambos escenarios agotan la CPU disponible. Sin margen libre, el costo de la telemetria ya no puede aparecer como mas consumo: se traduce en menos peticiones atendidas. Es la medicion del impacto en la capacidad del servicio, y corresponde al rango de 50-100 usuarios concurrentes que pide el enunciado.

- **Usuarios concurrentes:** 75 VUs (rampa de 30 s + meseta de 5m)
- **Peticiones linea base:** 17,044
- **Peticiones instrumentado:** 12,874
- **Limites por contenedor:** 1.0 CPU / 512 MiB (identicos en ambos escenarios)
- **Mezcla de trafico:** 85 % consulta de pedido (A → DB → B → DB), 10 % listado, 5 % pedido inexistente

Metrica	Sin OTel	Con OTel	Diferencia	Overhead
Latencia media	1158.79 ms	1644.85 ms	486.07 ms	+41.95 %
Latencia p95	1683.10 ms	2567.49 ms	884.38 ms	+52.54 %
Latencia p99	2584.18 ms	3222.66 ms	638.47 ms	+24.71 %
Throughput	47.50 req/s	35.79 req/s	-11.71 req/s	-24.65 %
CPU media de los servicios	119.86 %	122.70 %	2.84 pp	+2.37 %
CPU pico de los servicios	141.59 %	161.28 %	19.69 pp	+13.91 %
CPU del OTel Collector	3.29 %	5.22 %	1.93 pp	+58.62 %
Memoria pico de los servicios	175.39 MiB	324.70 MiB	149.32 MiB	+85.14 %

5.4 Interpretacion

El costo de la instrumentacion se reparte de forma desigual entre las tres dimensiones medidas, y conviene leer cada una con un criterio distinto:

Latencia. El camino critico de la peticion solo paga la creacion de spans en memoria y la escritura en la cola del `BatchSpanProcessor`. La exportacion por red ocurre en un hilo independiente, por lo que no bloquea la respuesta. El percentil p99 es el indicador mas informativo: la media puede esconder el costo real, mientras que la cola lenta revela el efecto de los vaciados periodicos del lote.

CPU. Es donde se concentra el gasto: serializar spans a Protobuf, agregar histogramas y formatear registros consume ciclos de forma proporcional al volumen de telemetria, no al numero de peticiones. El efecto solo es legible en el regimen no saturado; bajo saturacion queda oculto tras el techo del contenedor.

Memoria. El SDK introduce un costo *base fijo* —providers, exporters y buffers de los procesadores por lotes— que se paga al arrancar y que luego crece poco con la carga. En servicios pequenos este costo fijo puede representar un porcentaje llamativo sin ser preocupante en terminos absolutos.

Throughput. Es la traduccion practica del overhead cuando el servicio ya opera al limite. Cada ciclo de CPU dedicado a producir telemetria es un ciclo que no atiende peticiones, de modo que la capacidad maxima baja aunque el consumo medido se mantenga igual.

> **Sobre el lenguaje de la aplicacion.** Los servicios estan escritos en Python, donde el bloqueo global del interprete impide que el hilo de exportacion y el de atencion de peticiones progresen en paralelo dentro del mismo proceso. El overhead relativo medido aqui es, por tanto, un limite superior: una implementacion equivalente en Go o Java mostraria un costo menor con la misma configuracion de OpenTelemetry.

> **Palancas para reducir el overhead en produccion.**
> - **Muestreo.** Exportar el 100 % de las trazas es adecuado para un laboratorio, donde cada traza es evidencia. En produccion, `tail_sampling` permite conservar siempre los errores y las peticiones lentas, y muestrear solo el trafico exitoso rutinario.
> - **Cardinalidad de metricas.** Cada combinacion distinta de labels crea una serie temporal nueva. El procesador `attributes` elimina los atributos que no aportan valor analitico.
> - **Filtrado temprano.** Descartar los health checks evita procesar y almacenar un volumen alto de trazas sin informacion de negocio.
> - **Tamano del lote.** Lotes mayores reducen las llamadas de red a costa de mas memoria retenida; es un intercambio que se ajusta segun el perfil del servicio.

6. Decisiones de diseno

6.1 Dos readers de metricas

Las metricas salen por dos caminos simultaneos: *push* OTLP hacia el Collector —el camino de produccion— y un endpoint `/metrics` local para scraping directo. El segundo actua como control: permite comprobar que el Collector no pierde ni altera metricas en el trayecto. El efecto secundario es que cada serie aparece duplicada en Prometheus, distinguida por el label `source`; todas las consultas del dashboard filtran por `source="collector"`, sin lo cual el throughput se contaria dos veces.

6.2 Fronteras explicitas en los histogramas

Los buckets por defecto del SDK (`[0, 5, 10, 25, ... 10000]`) estan pensados para valores en milisegundos. Este laboratorio registra la latencia en segundos, siguiendo la convencion de Prometheus, de modo que practicamente todas las muestras caian en el primer bucket `[0, 5)` y `histogram_quantile` devolvia un p99 proximo a 5 s con independencia de la latencia real. Se definieron vistas con las fronteras estandar de latencia HTTP, lo que corrigio el calculo: el p99 observado paso de un valor artificial cercano a 5 s a decenas de milisegundos.

> Este fallo es un buen ejemplo de un riesgo caracteristico de la observabilidad: un panel que *parece* funcionar y muestra numeros plausibles puede estar midiendo mal. La deteccion no vino del dashboard, sino de contrastar el valor agregado con la latencia real observada en las trazas.

6.3 Pool de conexiones a base de datos

Sin un pool, cada peticion pagaria el handshake TCP y la autenticacion de PostgreSQL. Ese costo dominaria la medicion y se le atribuiria a OpenTelemetry una latencia que en realidad corresponde a la base de datos.

6.4 Jaeger y Tempo en paralelo

Jaeger aporta la interfaz de referencia del entregable. Tempo se integra de forma nativa con Grafana y su `metrics_generator` deriva metricas RED y el mapa de servicios a partir de las propias trazas, anadiendo una via adicional de correlacion entre trazas y metricas.

7. Reproducibilidad

```
# 1. Levantar el stack completo
docker compose up -d --build

# 2. Verificar la correlacion de las tres senales
bash scripts/verify-pipeline.sh

# 3. Ejecutar el benchmark comparativo
bash scripts/run-benchmark.sh

# 4. Regenerar este reporte con los resultados obtenidos
python docs/generar_reporte.py && bash scripts/generar-pdf.sh
```

Toda la configuracion esta versionada: dashboards como JSON, datasources como YAML de aprovisionamiento, infraestructura como codigo Terraform y dependencias con version fijada. No hay ningun paso manual en la interfaz de Grafana que haya que repetir para reconstruir el entorno.

8. Conclusiones

El pipeline cumple el objetivo planteado: las tres senales se emiten desde los microservicios, se centralizan en un unico Collector y resultan mutuamente navegables a traves del `trace_id`. La propagacion de contexto W3C entre servicios quedo verificada de forma automatizada, no solo observada en la interfaz.

Dos aprendizajes destacan por encima de la mecanica de configuracion. El primero es que **la correlacion es una decision de diseno, no una consecuencia automatica** de instrumentar: exige que el identificador viaje en las tres senales con un formato consistente y que las herramientas sepan como saltar entre ellas. El segundo es que **la telemetria tambien hay que verificarla**: el error en los buckets del histograma producia percentiles plausibles pero incorrectos, y un dashboard equivocado durante un incidente es peor que no tener dashboard, porque dirige la investigacion en la direccion equivocada.

Respecto al overhead, la conclusion practica es que el costo se concentra en CPU y en un consumo base fijo de memoria, mientras que el impacto en la latencia percibida por el usuario se mantiene acotado gracias a la exportacion asincrona por lotes. Ese costo es ajustable mediante muestreo y control de cardinalidad, lo que convierte la decision en un intercambio explicito entre profundidad de observabilidad y consumo de recursos, en lugar de un precio fijo que haya que aceptar sin mas.